Session 2:

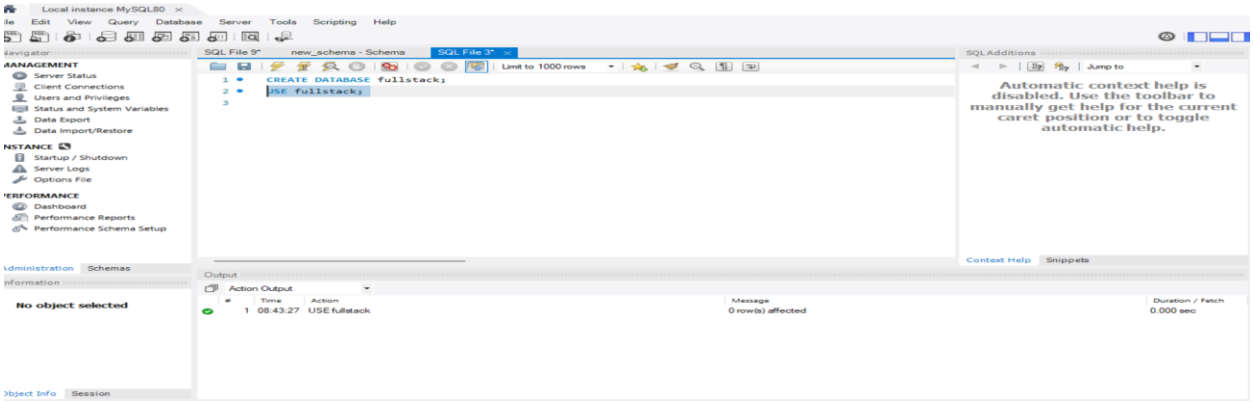
Install SQL tool, MySQL and VS Code.

1. Create a database:

CREATE DATABASE fullstack;

USE fullstack;

OUTPUT:



2. Create at least two tables:

Table 1: Student (VTU Number, Name, Email, Phone, Department)

CREATE TABLE Student (

VTU_Number VARCHAR(20) PRIMARY KEY,

Name VARCHAR(100),

Email VARCHAR(100),

Phone VARCHAR(15),

Department VARCHAR(50)

);

OUTPUT:

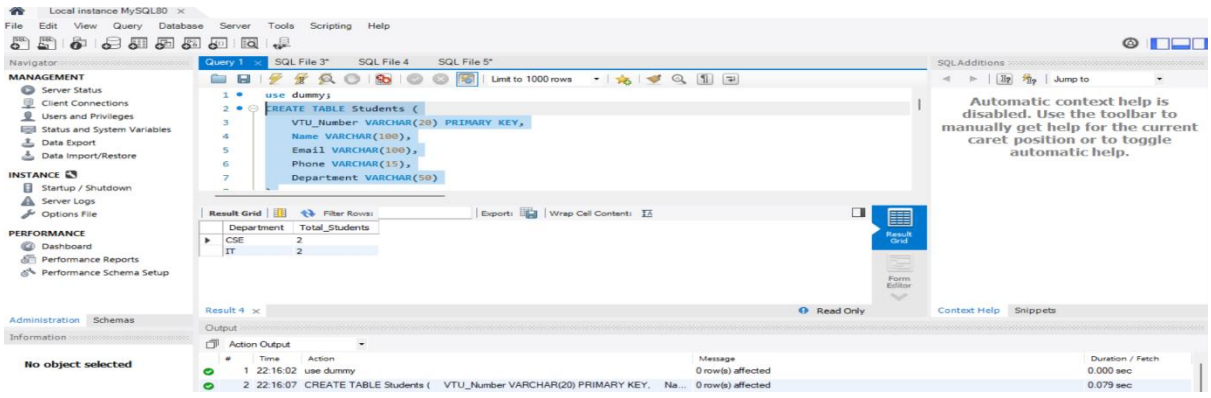


Table 2: Course (Course Code, Course Name, Faculty Id, Student ID, Faculty Email)

CREATE TABLE Course (

Course_Code VARCHAR(10) PRIMARY KEY,

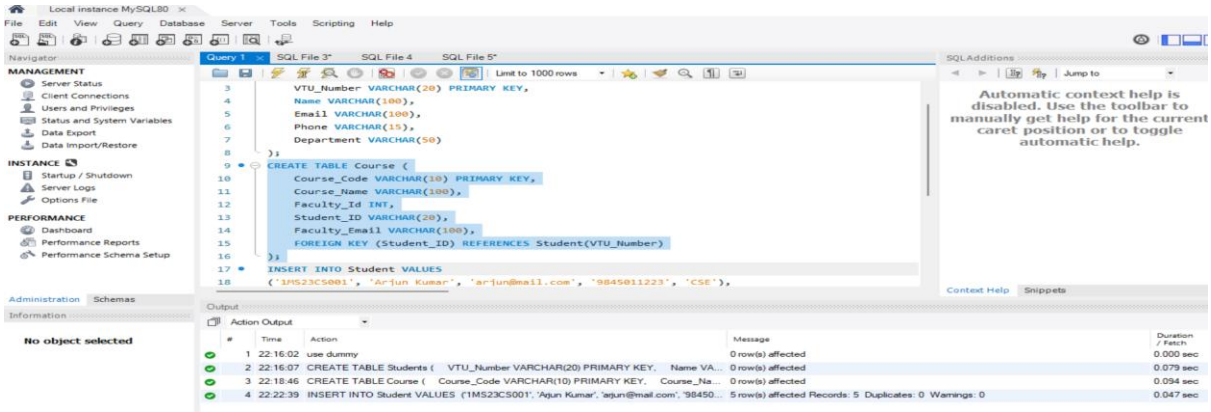
Course_Name VARCHAR(100),

Student_ID VARCHAR(20),

Faculty_Email VARCHAR(100),

FOREIGN KEY (Student_ID) REFERENCES Student(VTU_Number)

);OUTPUT:



3. Insert minimum 5 records into each table:

INSERT INTO Student VALUES

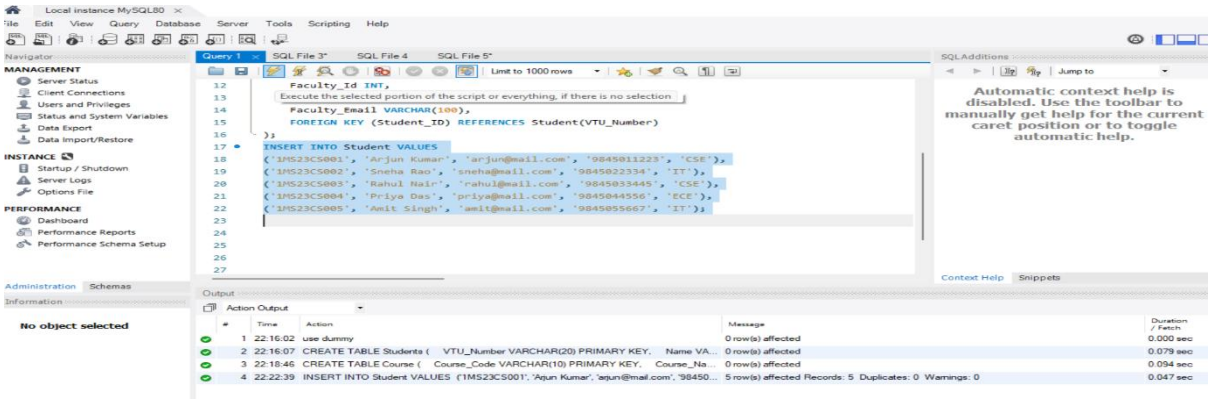
('1MS23CS001', 'Arjun Kumar', 'arjun@mail.com', '9845011223', 'CSE'),

('1MS23CS002', 'Sneha Rao', 'sneha@mail.com', '9845022334', 'IT'),

('1MS23CS004', 'Priya Das', 'priya@mail.com', '9845044556', 'ECE'),

('1MS23CS005', 'Amit Singh', 'amit@mail.com', '9845055667', 'IT');

OUTPUT:



INSERT INTO Course VALUES

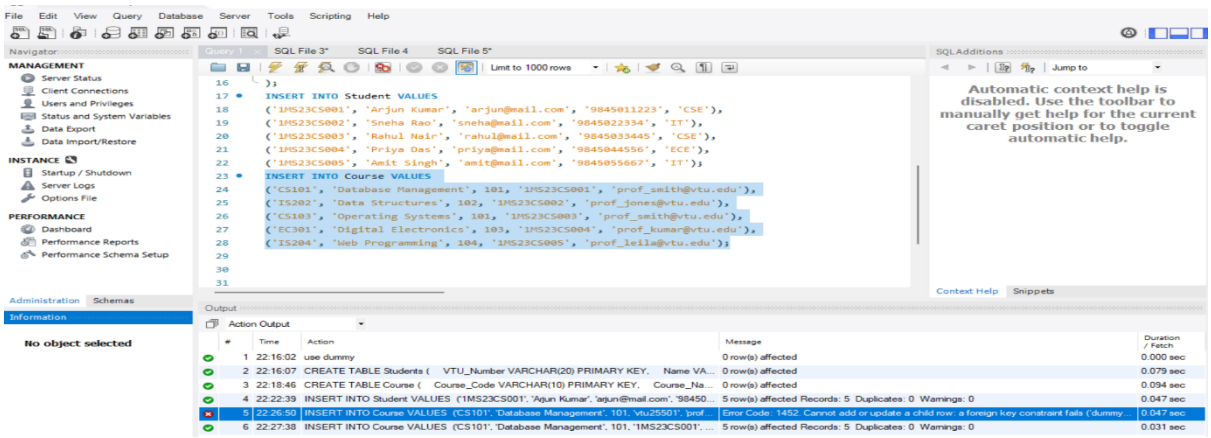
('CS101', 'Database Management', 101, '1MS23CS001', 'prof_smith@vtu.edu'),

('IS202', 'Data Structures', 102, '1MS23CS002', 'prof_jones@vtu.edu'),

('CS103', 'Operating Systems', 101, '1MS23CS003', 'prof_smith@vtu.edu'),

('EC301', 'Digital Electronics', 103, '1MS23CS004', 'prof_kumar@vtu.edu'),

OUTPUT:

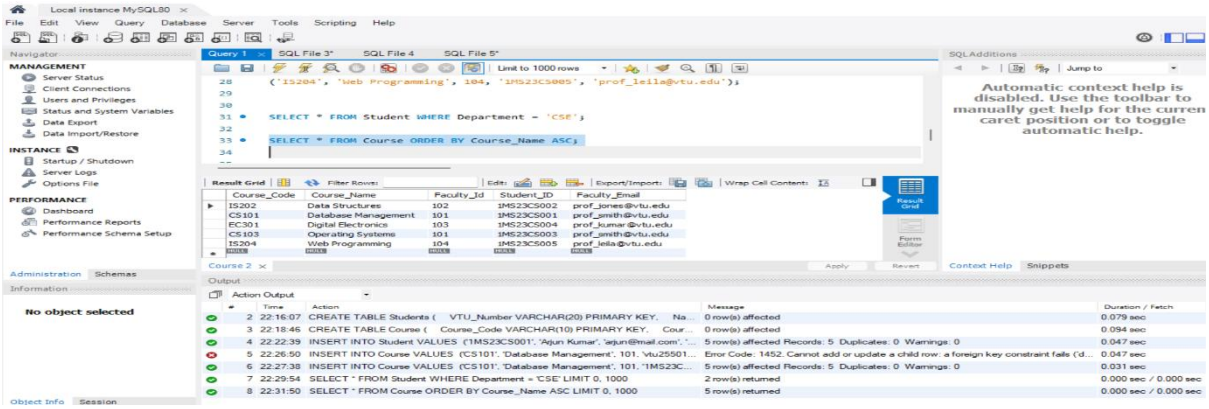


4. Select records using different CLAUSE:

   i. **WHERE clause:**

SELECT * FROM Student WHERE Department = 'CSE';

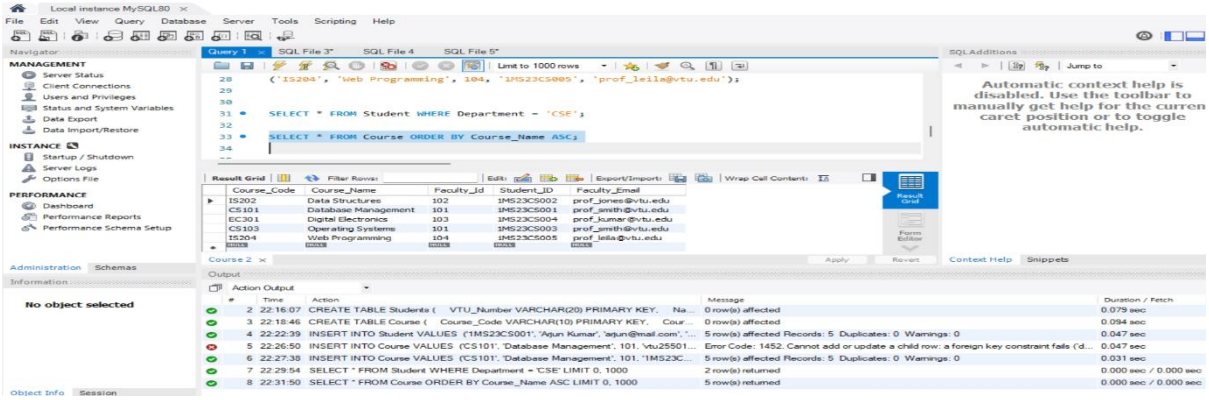
OUTPUT:



   ii. **ORDER BY Clause (Sorting):**

SELECT * FROM Course ORDER BY Course_Name ASC;

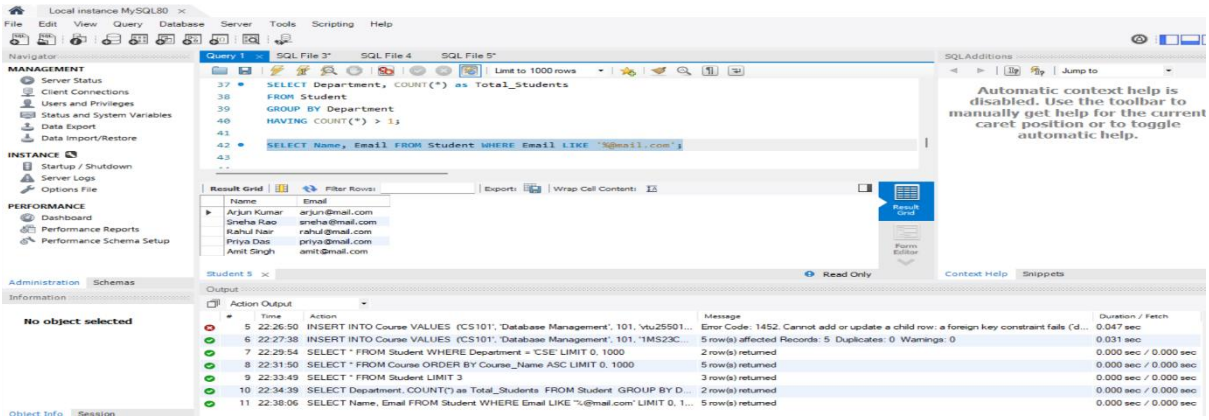
OUTPUT:



iii)**LIKE Clause (Pattern Matching):**

SELECT Name, Email FROM Student WHERE Email LIKE '%@mail.com';

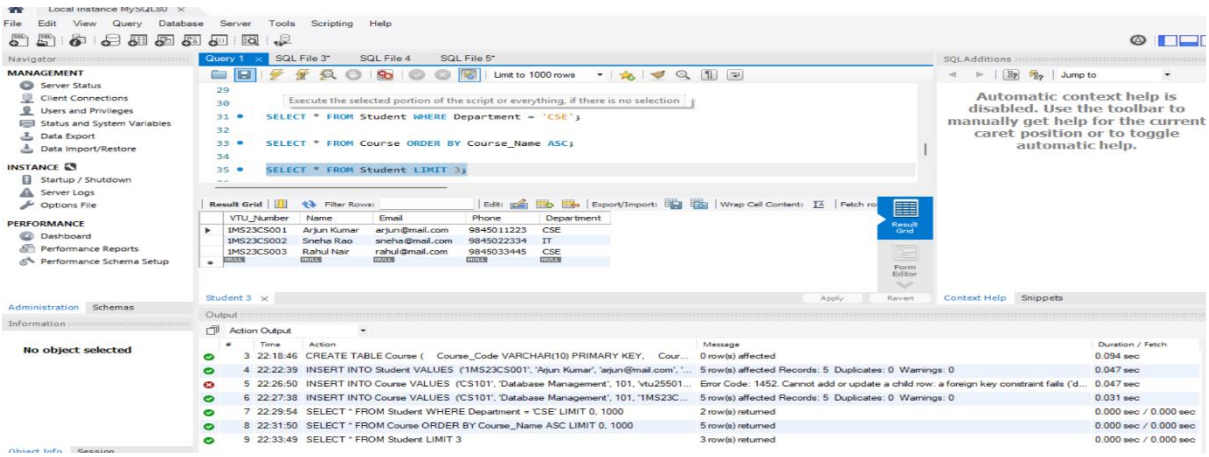
OUTPUT:



iii. **LIMIT Clause (Restricting Results):**

SELECT * FROM Student LIMIT 3;

OUTPUT:



iv)GROUP BY & HAVING Clause (Aggregating):

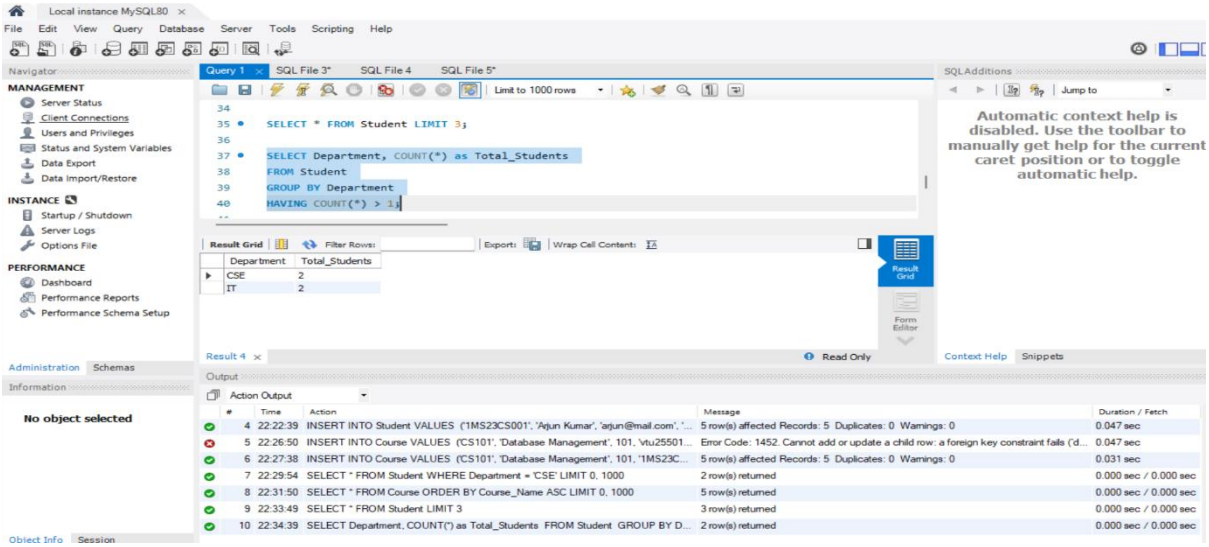
SELECT Department, COUNT(*) as Total_Students

FROM Student

GROUP BY Department

HAVING COUNT(*) > 1;

OUTPUT:



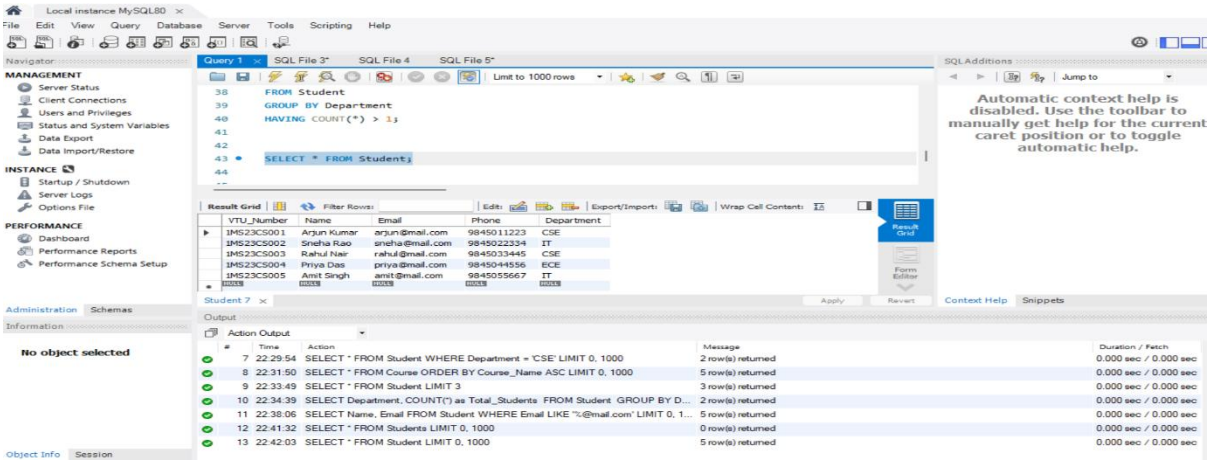
Session 3:

1. Write SELECT queries to display all records:

-- Display all students

SELECT * FROM Student;

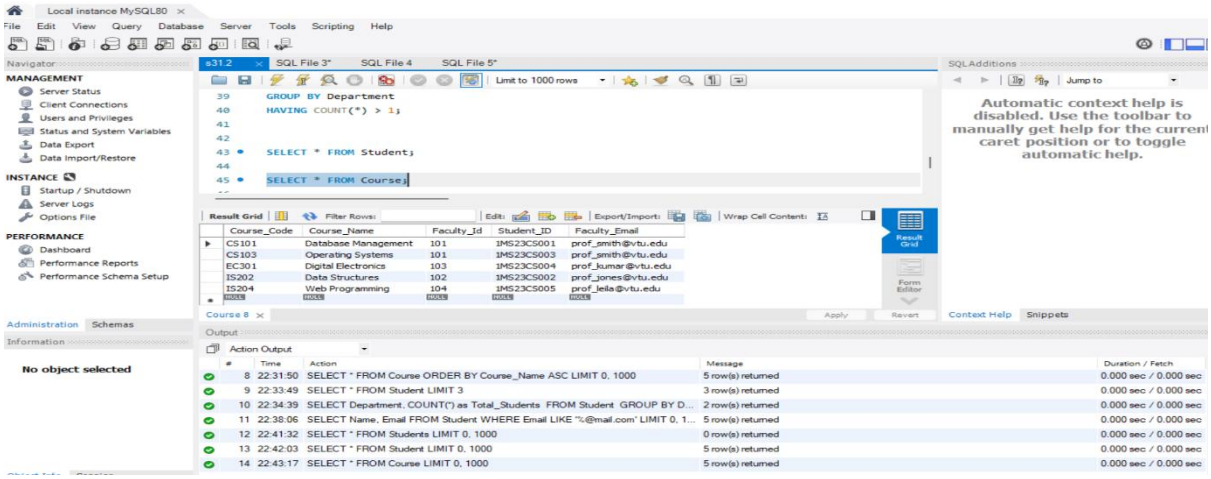
OUTPUT:



-- Display all courses

SELECT * FROM Course;

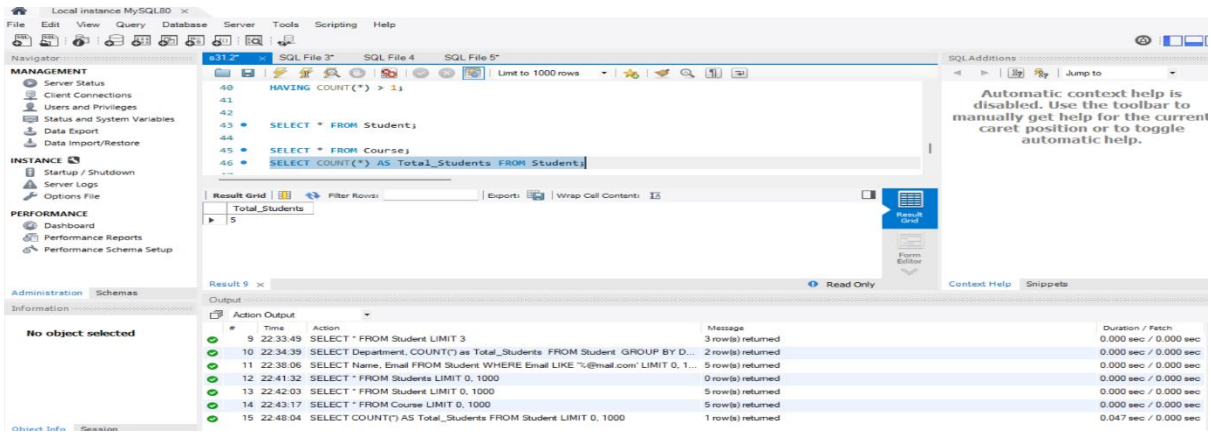
OUTPUT:



2)Write queries using aggregate functions:

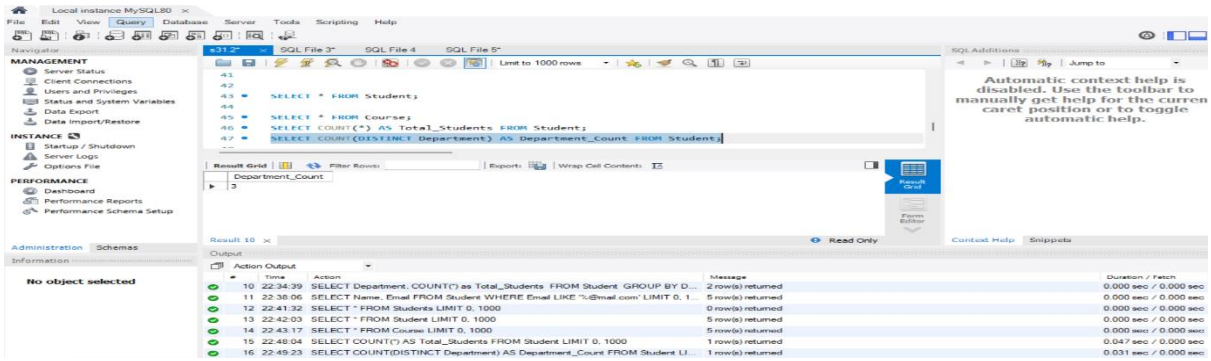
SELECT COUNT(*) AS Total_Students FROM Student;

OUTPUT:



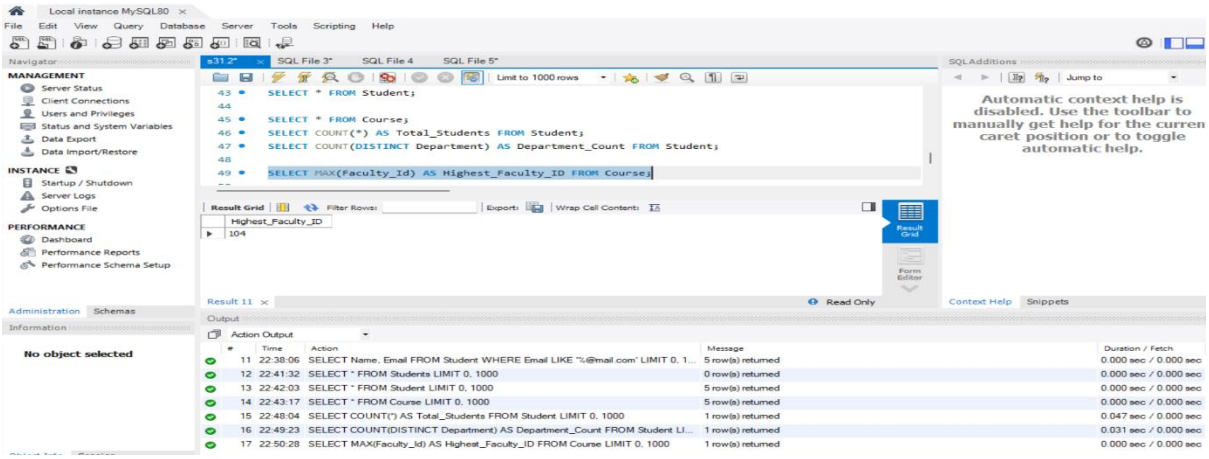
SELECT COUNT(DISTINCT Department) AS Department_Count FROM Student;

OUTPUT:



SELECT MAX(Faculty_Id) AS Highest_Faculty_ID FROM Course;

OUTPUT:



3)Sort data and display based in ascending/descending order of the VTU number:

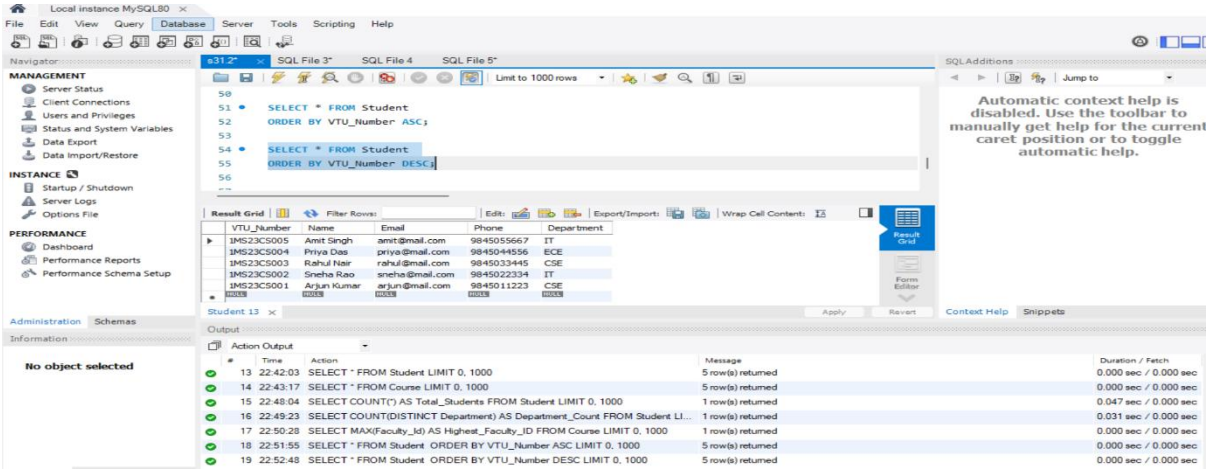
SELECT * FROM Student

ORDER BY VTU_Number ASC;

SELECT * FROM Student

ORDER BY VTU_Number DESC;

OUTPUT:

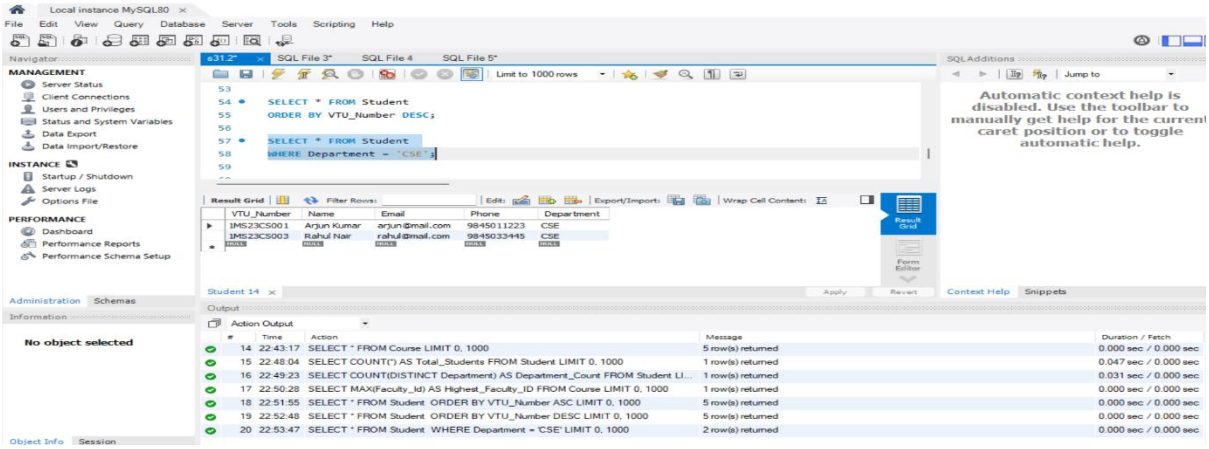


4)Display student records belonging to a particular department:

SELECT * FROM Student

WHERE Department = 'CSE';

OUTPUT:



5)Map VTU no. with course name and faculty using joins:

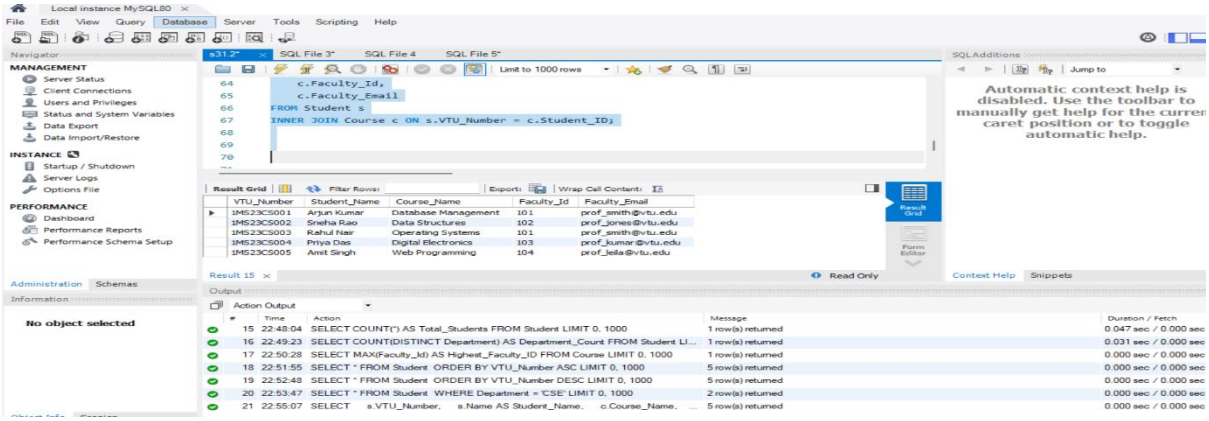
Joins Summary:

Join Type	Records Included
Inner	Only matches found in both tables.
Left	Everything from Left + matches from Right.
Right	Everything from Right + matches from Left.
Full	Everything from both tables.
Cross	Every possible combination of both tables.

Inner Join:

```
SELECT
  s.VTU_Number,
  s.Name AS Student_Name,
  c.Course_Name,
  c.Faculty_Id,
  c.Faculty_Email
FROM Student s
INNER JOIN Course c ON s.VTU_Number = c.Student_ID;
```

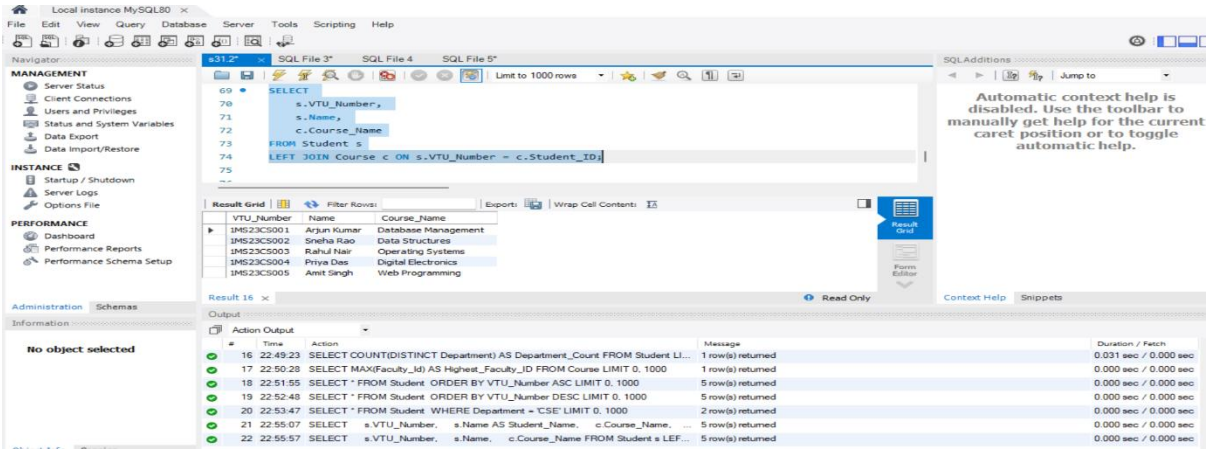
OUTPUT:



Left Join:

```
SELECT
    s.VTU_Number,
    s.Name,
    c.Course_Name
FROM Student s
LEFT JOIN Course c ON s.VTU_Number = c.Student_ID;
```

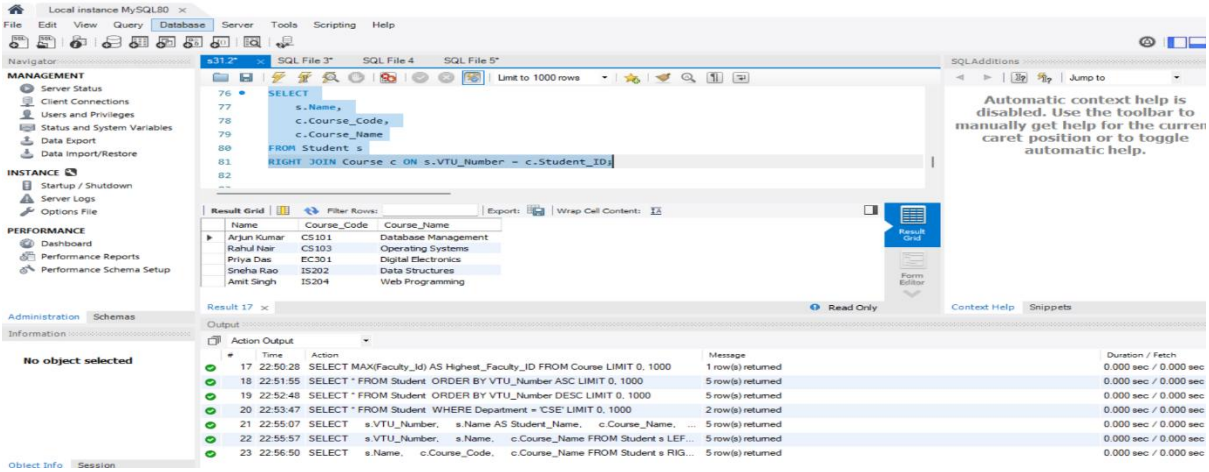
OUTPUT:



Right Join:

```
SELECT
    s.Name,
    c.Course_Code,
    c.Course_Name
FROM Student s
RIGHT JOIN Course c ON s.VTU_Number = c.Student_ID;
```

OUTPUT:



FULL JOIN (Full Outer Join):

SELECT s.Name, c.Course_Name FROM Student s

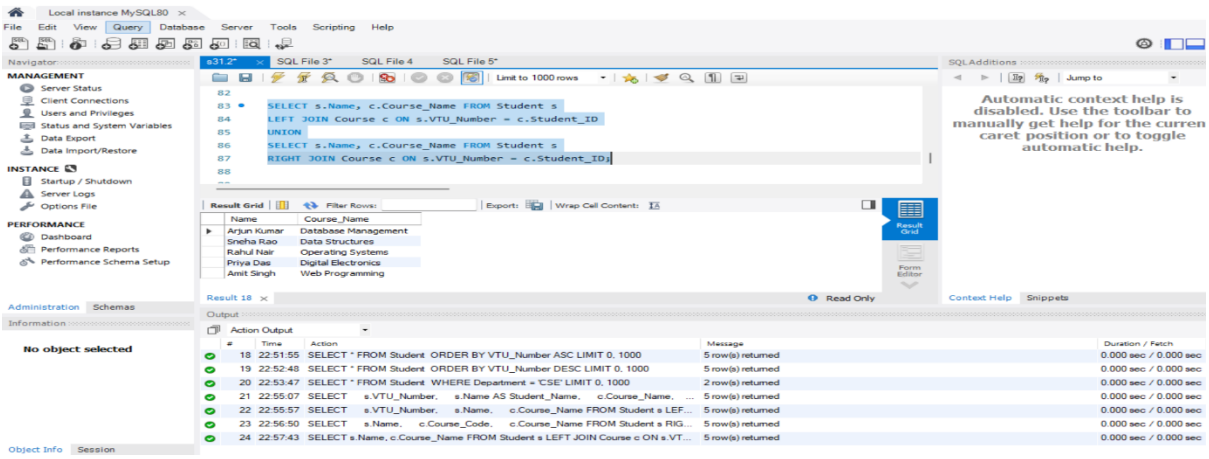
LEFT JOIN Course c ON s.VTU_Number = c.Student_ID

UNION

SELECT s.Name, c.Course_Name FROM Student s

RIGHT JOIN Course c ON s.VTU_Number = c.Student_ID;

OUTPUT:



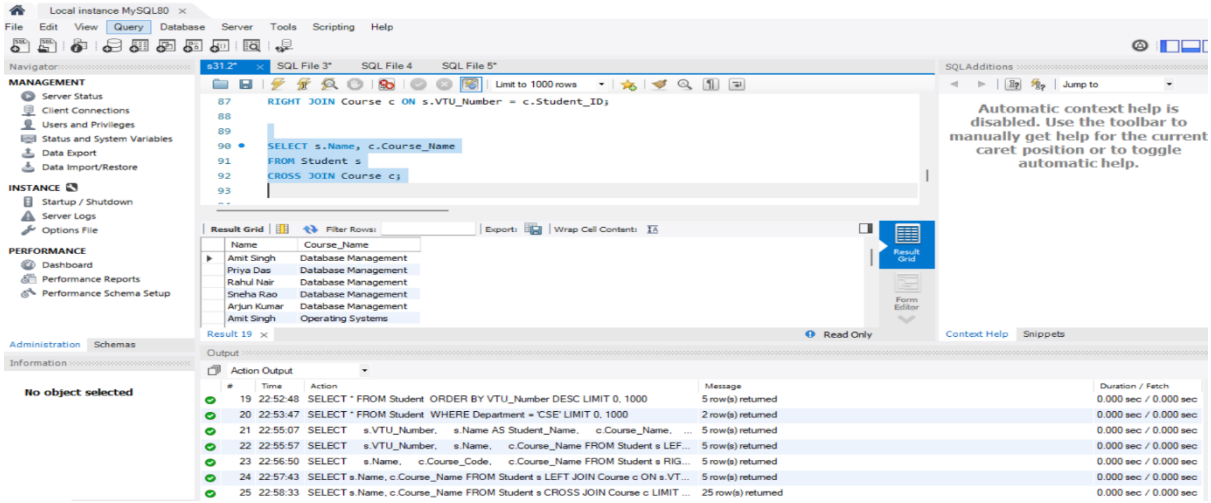
Cross Join(Cartesian Product):

SELECT s.Name, c.Course_Name

FROM Student s

CROSS JOIN Course c;

OUTPUT:



Session 4: SQL

1. Select set of students based on course and count the number of students:

SELECT

c.Course_Name,

COUNT(s.VTU_Number) AS Total_Students

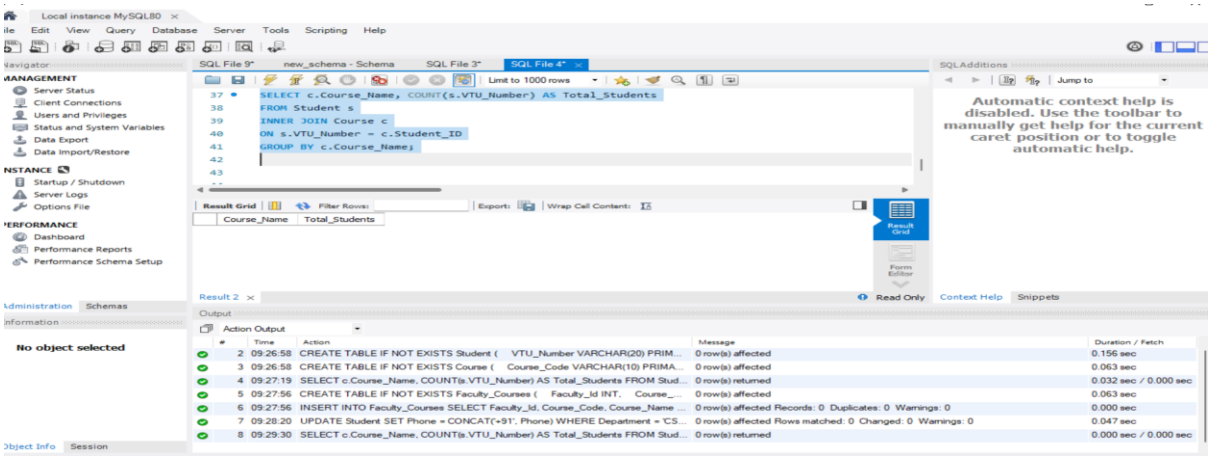
FROM Student s

INNER JOIN Course c

ON s.VTU_Number = c.Student_ID

GROUP BY c.Course_Name;

OUTPUT:



2. Select set of courses a faculty is handling and insert into the new table:

CREATE TABLE Faculty_Courses (

Faculty_Id INT,

Course_Code VARCHAR(10),

Course_Name VARCHAR(100)

);

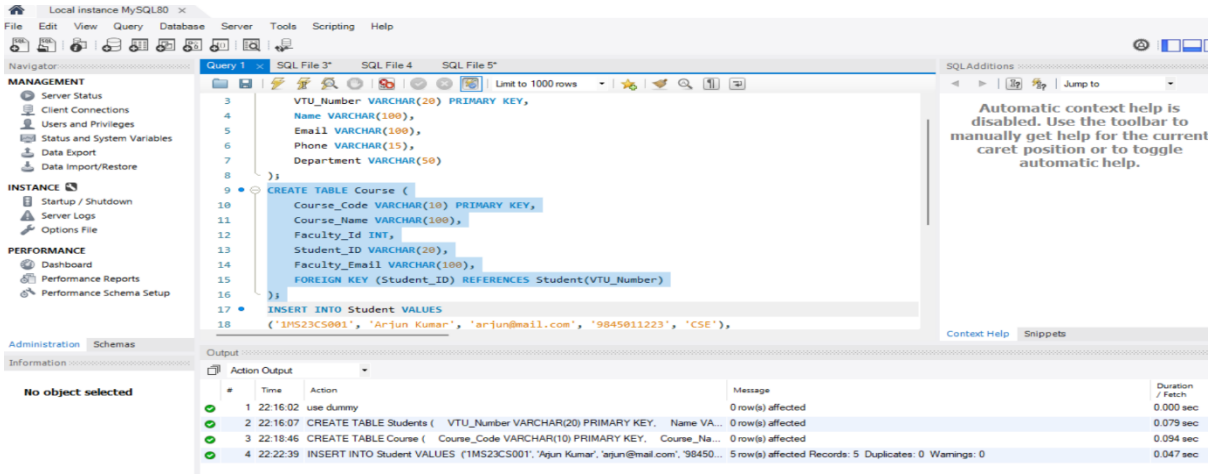
INSERT INTO Faculty_Courses

SELECT Faculty_Id, Course_Code, Course_Name

FROM Course

WHERE Faculty_Id = 101;

OUTPUT:



3. Select top 5 students from same department and update their phone no:

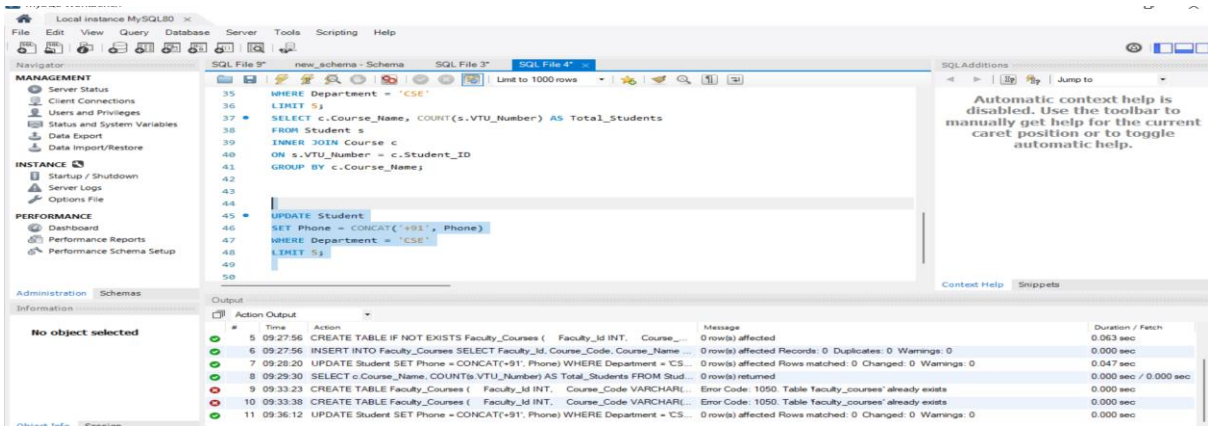
UPDATE Student

SET Phone = CONCAT('+91', Phone)

WHERE Department = 'CSE'

LIMIT 5;

OUTPUT:



Session 5 : SQL

1)Initiate and commit a transaction:

START TRANSACTION;

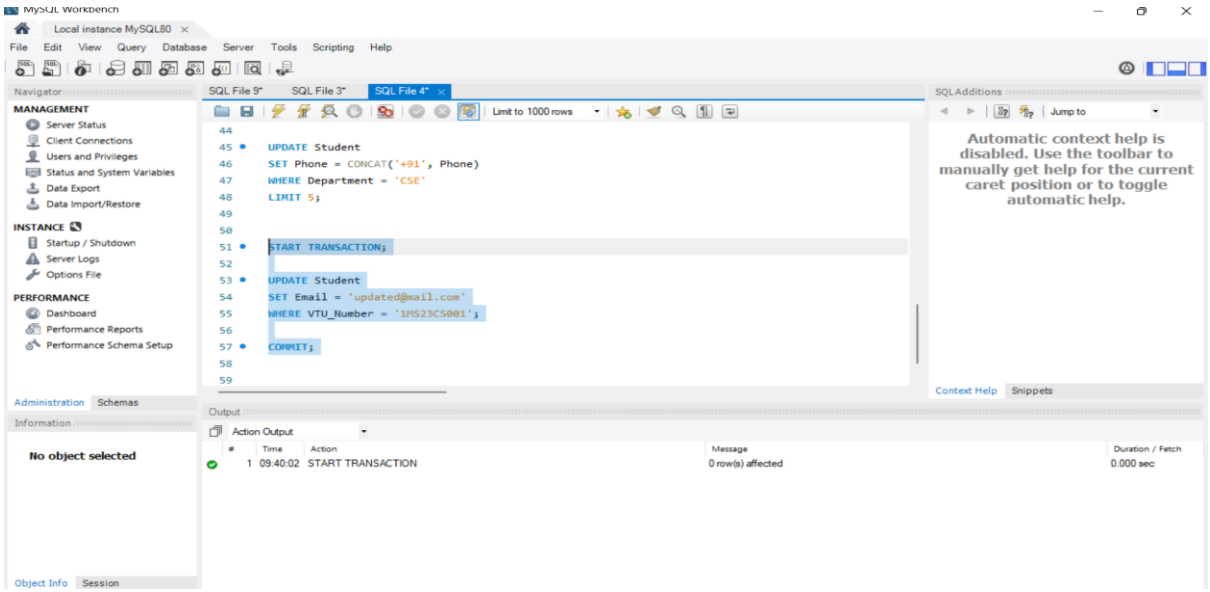
UPDATE Student

SET Email = 'updated@mail.com'

WHERE VTU_Number = '1MS23CS001';

COMMIT;

OUTPUT:



2)Savepoint and rollbac:

START TRANSACTION;

UPDATE Student

SET Phone = '9999999999'

WHERE VTU_Number = '1MS23CS002';

SAVEPOINT sp1;

UPDATE Student

SET Phone = '8888888888'

WHERE VTU_Number = '1MS23CS003';

ROLLBACK TO sp1;

COMMIT;

3)Create a user-defined function (math):

USE fullstack;

CREATE FUNCTION add_numbers(a INT, b INT)

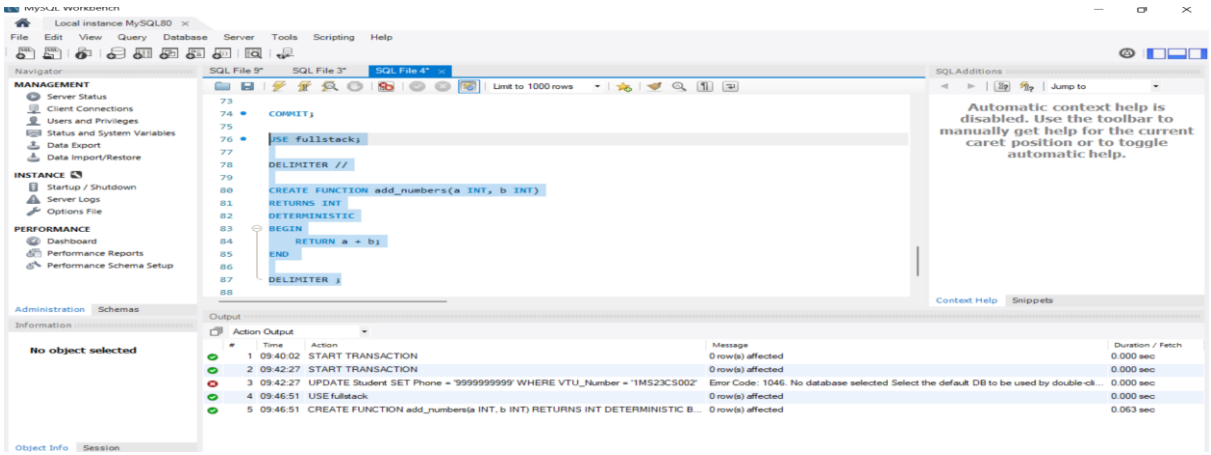
RETURNS INT

DETERMINISTIC

BEGIN

RETURN a + b;

OUTPUT:



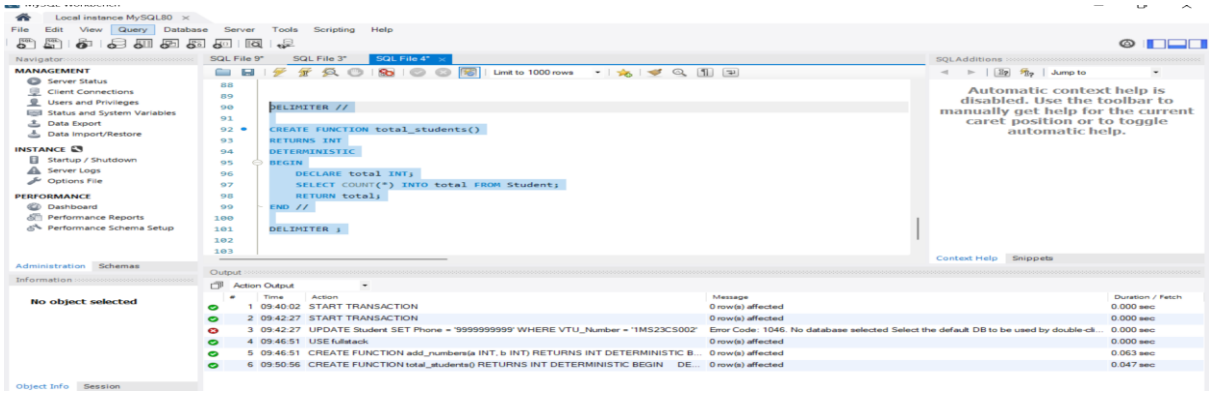
4)Create a user-defined function to execute a SELECT query to return a value:

CREATE FUNCTION total_students()

RETURNS INT

DETERMINISTICBEGIN

DECLARE total INT;

SELECT COUNT(*) INTO total FROM Student;

OUTPUT: